

TEORETICKÁ INFORMATIKA

Binární halda

David Weber

weber3@spsejecna.cz



Rok 2025/26

Binární stromy

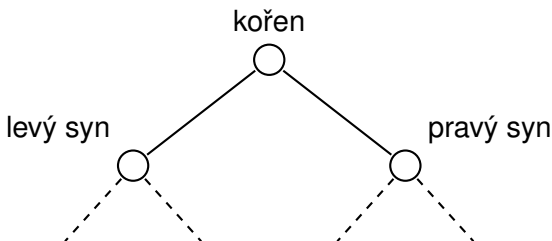
Definice (Binární strom)

Strom T nazveme **binární**, pokud je **zakořeněný** a každý vrchol (rodič) má maximálně **dva syny**, u nichž rozlišujeme, který je levý a který pravý.

Binární strom

Definice (Binární strom)

Strom T nazveme **binární**, pokud je **zakořeněný** a každý vrchol (rodič) má maximálně **dva syny**, u nichž rozlišujeme, který je levý a který pravý.



Hladiny v binárním stromě

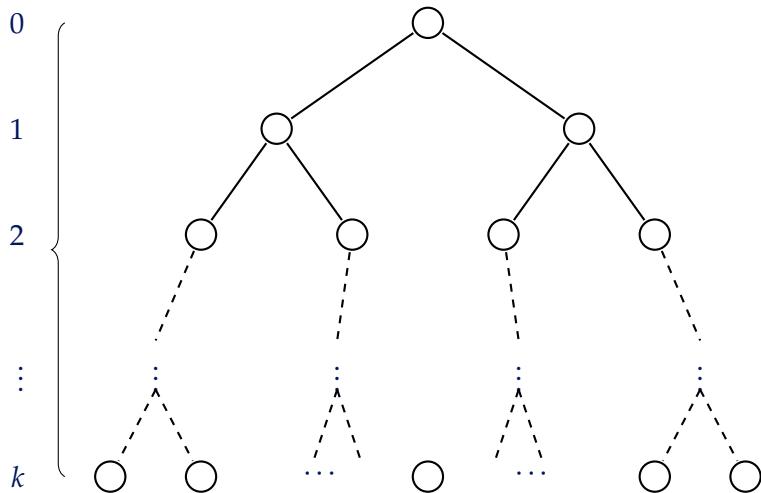
Definice (Hladina)

Nechť $T = (V, E)$ je binární strom, v_0 jeho kořen a číslo $k \in \mathbb{N}_0$. Pak k -tou hladinou stromu T rozumíme množinu vrcholů

$$L_k = \{v \in V : d(v, v_0) = k\}.$$

Tedy hladina L_k je množina všech vrcholů, které mají stejnou vzdálenost od kořene.

Znázornění hladin



Binární halda

Příklad na začátek . . .

Příklad na začátek . . .

- Máme zadáný soubor čísel, v němž chceme rychle najít **minimum**, resp. **maximum**.

Příklad na začátek ...

- Máme zadaný soubor čísel, v němž chceme rychle najít **minimum**, resp. **maximum**.
- Prvky lze uložit do seznamu/pole.

Příklad na začátek ...

- Máme zadáný soubor čísel, v němž chceme rychle najít **minimum**, resp. **maximum**.
- Prvky lze uložit do seznamu/pole.

Algoritmus: Minimum

Vstup: Seznam prvků $x_1, x_2, \dots, x_n$

```
1 min ←  $x_1$ ;  
2 for  $i = 1, 2, \dots, n$  do  
3   | if min <  $x_i$  then min ←  $x_i$ ;  
4 return min
```

Příklad na začátek ...

- Máme zadáný soubor čísel, v němž chceme rychle najít **minimum**, resp. **maximum**.
- Prvky lze uložit do seznamu/pole.

Algoritmus: Minimum

Vstup: Seznam prvků $x_1, x_2, \dots, x_n$

```
1  $\text{min} \leftarrow x_1$ ;  
2 for  $i = 1, 2, \dots, n$  do  
3   if  $\text{min} < x_i$  then  $\text{min} \leftarrow x_i$ ;  
4 return  $\text{min}$ 
```

- Časová složitost bude zjevně lineární, tj. $O(n)$.

Příklad na začátek ...

- Máme zadaný soubor čísel, v němž chceme rychle najít **minimum**, resp. **maximum**.
- Prvky lze uložit do seznamu/pole.

Algoritmus: Minimum

Vstup: Seznam prvků $x_1, x_2, \dots, x_n$

```
1  $\text{min} \leftarrow x_1$ ;  
2 for  $i = 1, 2, \dots, n$  do  
3   if  $\text{min} < x_i$  then  $\text{min} \leftarrow x_i$ ;  
4 return min
```

- Časová složitost bude zjevně lineární, tj. $O(n)$.
- Rychlejšího algoritmu lze dosáhnout pomocí tzv. **binární haldy**.

Minimová binární halda

Minimová binární halda

- Budeme pracovat s binárním stromem $T = (V, E)$.

Minimová binární halda

- Budeme pracovat s binárním stromem $T = (V, E)$.
- Každému vrcholu v přiřadíme tzv. **klíč**, který bude v našem případě představovat libovolné celé číslo. Tuto hodnotu budeme značit $\text{key}(v)$.

Minimová binární halda

- Budeme pracovat s binárním stromem $T = (V, E)$.
- Každému vrcholu v přiřadíme tzv. **klíč**, který bude v našem případě představovat libovolné celé číslo. Tuto hodnotu budeme značit $\text{key}(v)$.

Formálně se jedná o zobrazení $\text{key} : V \rightarrow \mathbb{Z}$.

Minimová binární halda

- Budeme pracovat s binárním stromem $T = (V, E)$.
- Každému vrcholu v přiřadíme tzv. **klíč**, který bude v našem případě představovat libovolné celé číslo. Tuto hodnotu budeme značit $\text{key}(v)$.

Formálně se jedná o zobrazení $\text{key} : V \rightarrow \mathbb{Z}$.

Definice (Minimová binární halda)

Minimová binární halda je datová struktura tvaru binárního stromu, v jehož každém vrcholu je uložen jeden prvek. Přitom platí:

- 1 **Tvar haldy:** Všechny hladiny kromě poslední jsou plně obsazené. Poslední hladina je zaplněna zleva.
- 2 **Haldové uspořádání:** Je-li v vrchol a s jeho syn, pak platí $\text{key}(v) \leq \text{key}(s)$.

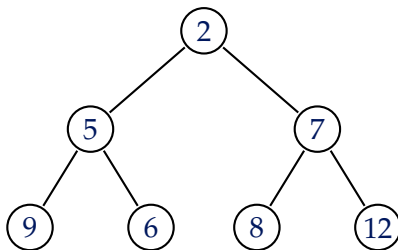
Definice (Maximová binární halda)

Minimová binární halda je datová struktura tvaru binárního stromu, v jehož každém vrcholu je uložen jeden prvek. Přitom platí:

- 1 **Tvar haldy:** Všechny hladiny kromě poslední jsou plně obsazené. Poslední hladina je zaplněna zleva.
- 2 **Haldové uspořádání:** Je-li v vrchol a s jeho syn, pak platí $\text{key}(v) \geq \text{key}(s)$.

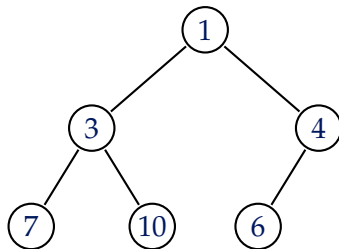
Minimová binární halda

Příklad I



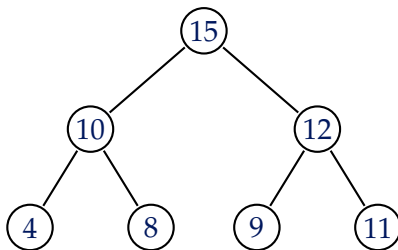
Minimová binární halda

Příklad II



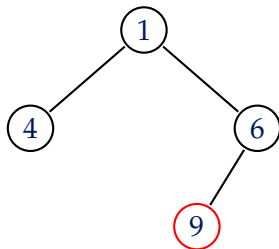
Maximová binární halda

Příklad



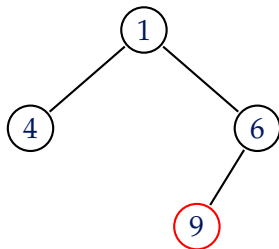
Minimová binární halda

Protipříklad I



Minimová binární halda

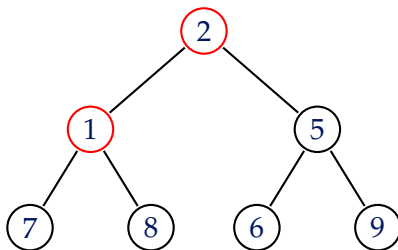
Protipříklad I



Strom porušuje podmínku na **tvar haldy**.

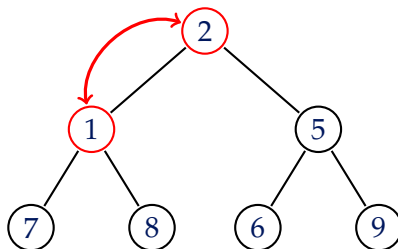
Minimová binární halda

Protipříklad II



Minimová binární halda

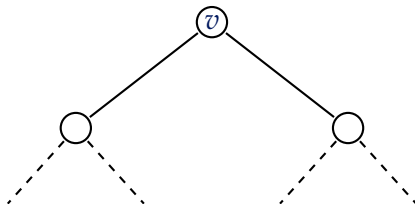
Protipříklad II



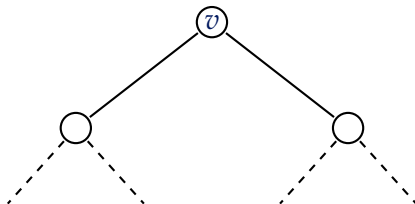
Strom porušuje **haldové uspořádání**.

Pozorování

Pozorování

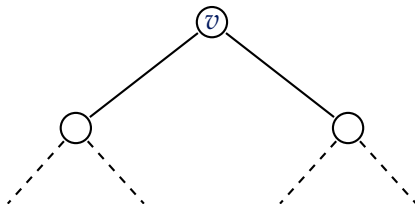


Pozorování



V kořeni je vždy uložen **vrchol s minimálním klíčem**.
Podobně v maximové binární haldě je v kořeni vždy
uloženo maximum.

Pozorování



V kořeni je vždy uložen **vrchol s minimálním klíčem**.
Podobně v maximové binární haldě je v kořeni vždy
uloženo maximum.

To má jednu zřejmou výhodu: nalezení minima/maxima v
takové struktuře je **velmi rychlé**.

Operace s minimovou binární haldou

To nám ale nestačí! Haldu musíme nějak udržovat.

Operace s minimovou binární haldou

To nám ale nestačí! Haldu musíme nějak udržovat.

Rádi bychom v haldě uměli provádět následující operace:

Operace s minimovou binární haldou

To nám ale nestačí! Haldu musíme nějak udržovat.

Rádi bychom v haldě uměli provádět následující operace:

- 1 **MIN** = vrátí minimální hodnotu klíče uloženou v haldě.

Operace s minimovou binární haldou

To nám ale nestačí! Haldu musíme nějak udržovat.

Rádi bychom v haldě uměli provádět následující operace:

- 1 **MIN** = vrátí minimální hodnotu klíče uloženou v haldě.
- 2 **INSERT**(x) = vloží nový prvek x do haldy.

Operace s minimovou binární haldou

To nám ale nestačí! Haldu musíme nějak udržovat.

Rádi bychom v haldě uměli provádět následující operace:

- 1 **MIN** = vrátí minimální hodnotu klíče uloženou v haldě.
- 2 **INSERT**(x) = vloží nový prvek x do haldy.
- 3 **EXTRACTMIN** = odebere vrchol s nejmenším klíčem a vrátí jej.

Operace s minimovou binární haldou

To nám ale nestačí! Haldu musíme nějak udržovat.

Rádi bychom v haldě uměli provádět následující operace:

- 1 **MIN** = vrátí minimální hodnotu klíče uloženou v haldě.
- 2 **INSERT**(x) = vloží nový prvek x do haldy.
- 3 **EXTRACTMIN** = odebere vrchol s nejmenším klíčem a vrátí jej.

V případě **maximové binární haldy** bude situace obdobná – budeme chtít operace **MAX** a **EXTRACTMAX**

Počet hladin haldy

Počet hladin haldy

Tvrzení

Binární halda s n prvky má $\lceil \log_2(n + 1) \rceil$ hladin.

Počet hladin haldy

Tvrzení

Binární halda s n prvky má $\lceil \log_2(n + 1) \rceil$ hladin.

- Předpokládejme, že uvažovaná halda má všechny hladiny plně obsazené. Jejich počet si označme h a počet vrcholů n .

Počet hladin haldy

Tvrzení

Binární halda s n prvky má $\lceil \log_2(n + 1) \rceil$ hladin.

- Předpokládejme, že uvažovaná halda má všechny hladiny plně obsazené. Jejich počet si označme h a počet vrcholů n .
- Pak počet vrcholů lze spočítat jako

$$n = 2^0 + 2^1 + 2^2 + \dots + 2^{h-1} = 2^h - 1.$$

Počet hladin haldy

Tvrzení

Binární halda s n prvky má $\lceil \log_2(n + 1) \rceil$ hladin.

- Předpokládejme, že uvažovaná halda má všechny hladiny plně obsazené. Jejich počet si označme h a počet vrcholů n .
- Pak počet vrcholů lze spočítat jako

$$n = 2^0 + 2^1 + 2^2 + \dots + 2^{h-1} = 2^h - 1.$$

- Úpravou získáme

$$n = 2^h - 1 \iff n + 1 = 2^h \iff h = \log_2(n + 1).$$

Počet hladin haldy

Tvrzení

Binární halda s n prvky má $\lceil \log_2(n + 1) \rceil$ hladin.

- Předpokládejme, že uvažovaná halda má všechny hladiny plně obsazené. Jejich počet si označme h a počet vrcholů n .
- Pak počet vrcholů lze spočítat jako

$$n = 2^0 + 2^1 + 2^2 + \dots + 2^{h-1} = 2^h - 1.$$

- Úpravou získáme

$$n = 2^h - 1 \iff n + 1 = 2^h \iff h = \log_2(n + 1).$$

- Pokud poslední hladina není zcela zaplněná, bude pak nebude h celé číslo. To lze „opravit“ přidáním zaokrouhlení nahoru:

$$h = \lceil \log_2(n + 1) \rceil.$$

Operace **INSERT** – vkládání vrcholu

Operace **INSERT** – vkládání vrcholu

- Prázdnou nebo jednoprvkovou haldu lze vytvořit snadno.

Operace **INSERT** – vkládání vrcholu

- Prázdnou nebo jednoprvkovou haldu lze vytvořit snadno.
- Uvažujme, že do obecné (minimové) haldy chceme přidat nový prvek.

Operace **INSERT** – vkládání vrcholu

- Prázdnou nebo jednoprvkovou haldu lze vytvořit snadno.
- Uvažujme, že do obecné (minimové) haldy chceme přidat nový prvek.
- Z podmínky na **tvar haldy** plyne, že nový vrchol buď

Operace **INSERT** – vkládání vrcholu

- Prázdnou nebo jednoprvkovou haldu lze vytvořit snadno.
- Uvažujme, že do obecné (minimové) haldy chceme přidat nový prvek.
- Z podmínky na **tvár haldy** plyne, že nový vrchol buď
 - přidáme na konec poslední hladiny, nebo

Operace **INSERT** – vkládání vrcholu

- Prázdnou nebo jednoprvkovou haldu lze vytvořit snadno.
- Uvažujme, že do obecné (minimové) haldy chceme přidat nový prvek.
- Z podmínky na **tvár haldy** plyne, že nový vrchol buď
 - přidáme na konec poslední hladiny, nebo
 - založíme novou hladinu a vrchol přidáme úplně vlevo, pokud je poslední hladina plná.

Operace **INSERT** – vkládání vrcholu

- Prázdnou nebo jednoprvkovou haldu lze vytvořit snadno.
- Uvažujme, že do obecné (minimové) haldy chceme přidat nový prvek.
- Z podmínky na **tvar haldy** plyne, že nový vrchol buď
 - přidáme na konec poslední hladiny, nebo
 - založíme novou hladinu a vrchol přidáme úplně vlevo, pokud je poslední hladina plná.

Problém: Tímto krokem může dojít k porušení haldového uspořádání, neboť pro nový vrchol v a jeho rodiče r může nastat, že $\text{key}(v) < \text{key}(r)$.

Procedura BUBBLEUP

- Po vložení nového vrcholu musíme haldu upravit tak, aby platilo haldové uspořádání, není-li splněno.

- Po vložení nového vrcholu musíme haldu upravit tak, aby platilo haldové uspořádání, není-li splněno.
- Jako první prohodíme nový vrchol v s jeho rodičem r , pokud $\text{key}(v) < \text{key}(r)$.

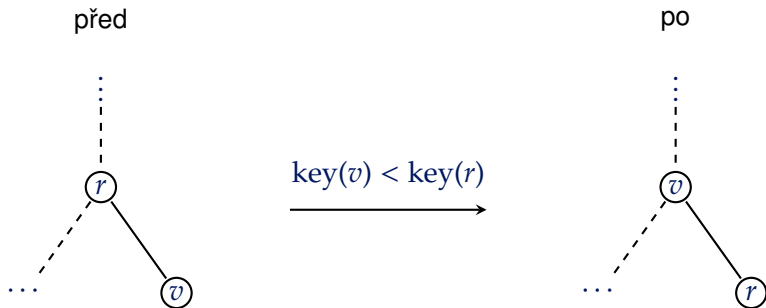
- Po vložení nového vrcholu musíme haldu upravit tak, aby platilo haldové uspořádání, není-li splněno.
- Jako první prohodíme nový vrchol v s jeho rodičem r , pokud $\text{key}(v) < \text{key}(r)$.
- Prohozením vrcholů jsme opravili uvedenou nerovnost, ale mohli jsme způsobit chybu o hladinu výš.

- Po vložení nového vrcholu musíme haldu upravit tak, aby platilo haldové uspořádání, není-li splněno.
- Jako první prohodíme nový vrchol v s jeho rodičem r , pokud $\text{key}(v) < \text{key}(r)$.
- Prohozením vrcholů jsme opravili uvedenou nerovnost, ale mohli jsme způsobit chybu o hladinu výš.
- V takovém případě postup opakujeme pro vrchol v a jeho rodičem r' atd.

Procedura BUBBLEUP

- Po vložení nového vrcholu musíme haldu upravit tak, aby platilo haldové uspořádání, není-li splněno.
- Jako první prohodíme nový vrchol v s jeho rodičem r , pokud $\text{key}(v) < \text{key}(r)$.
- Prohozením vrcholů jsme opravili uvedenou nerovnost, ale mohli jsme způsobit chybu o hladinu výš.
- V takovém případě postup opakujeme pro vrchol v a jeho rodičem r' atd.
- Taková procedura se někdy nazývá BUBBLEUP, neboť vrchol postupně „probublá“ nahoru (potenciálně až do kořene, kde se procedura zastaví).

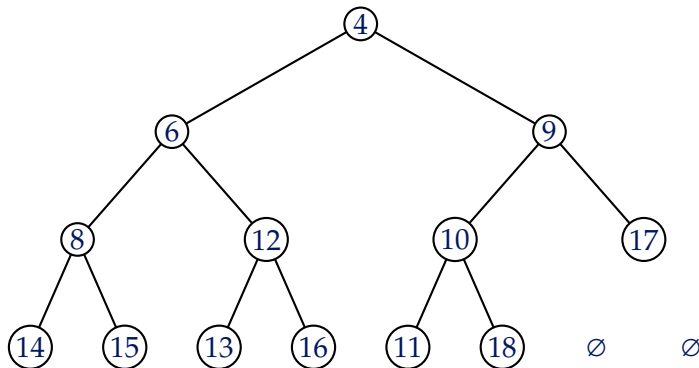
Znázornění BUBBLEUP



Procedura BUBBLEUP

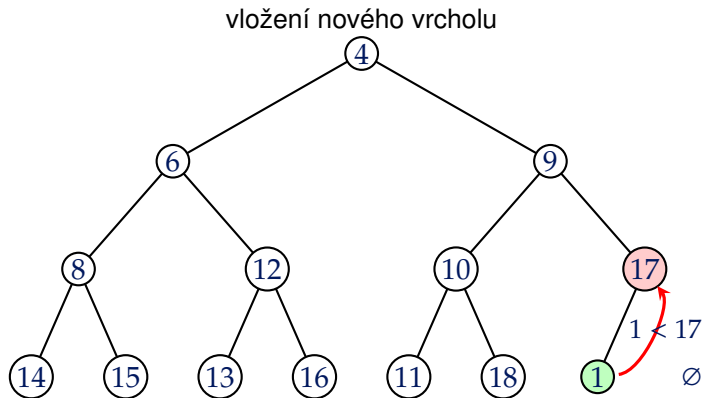
1. krok

původní minimová halda



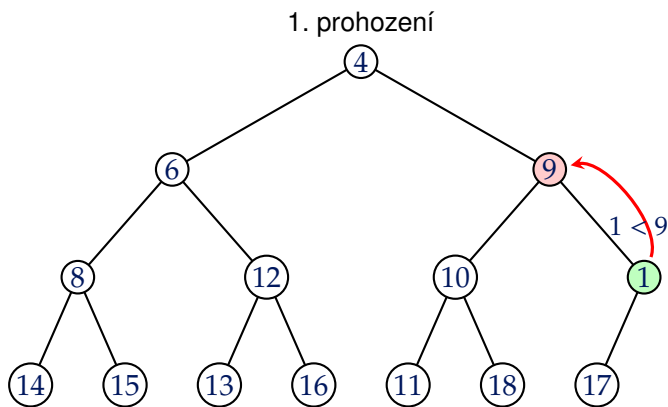
Procedura BUBBLEUP

2. krok



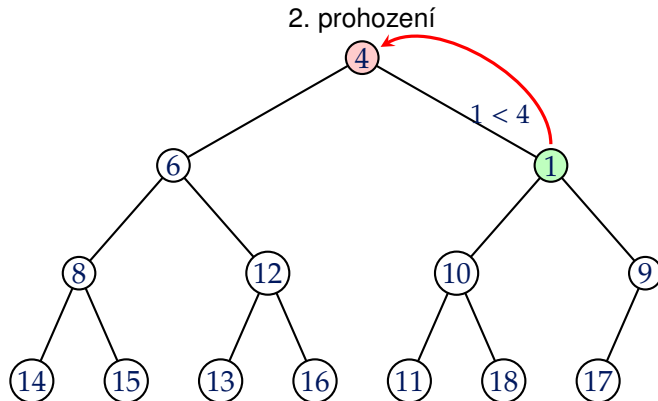
Procedura BUBBLEUP

3. krok



Procedura BUBBLEUP

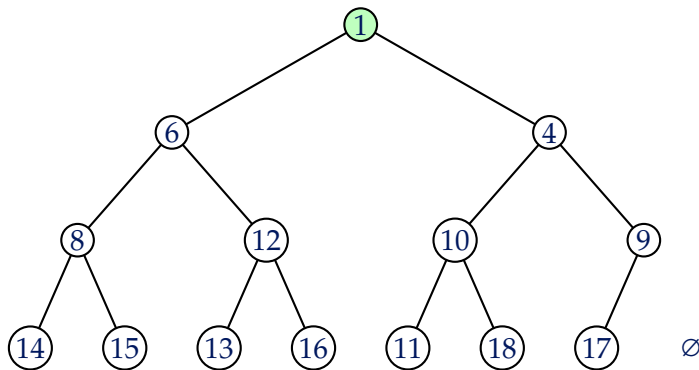
4. krok



Procedura BUBBLEUP

5. krok

výsledná minimová halda



Algoritmus: BUBBLEUP (oprava haldy směrem nahorů)

Vstup: Vrchol v v minimové binární haldě

- 1 **while** v má rodiče r a $\text{key}(v) < \text{key}(r)$ **do**
 - 2 prohod' klíče ve vrcholech v a r ;
 - 3 $v \leftarrow r$;
-

Algoritmus: BUBBLEUP (oprava haldy směrem nahorů)

Vstup: Vrchol v v minimové binární haldě

- 1 **while** v má rodiče r a $\text{key}(v) < \text{key}(r)$ **do**
 - 2 prohod' klíče ve vrcholech v a r ;
 - 3 $v \leftarrow r$;
-

V případě maximové binární haldy bychom zkrátka jen uvedli opačnou nerovnost:

$$\text{key}(v) > \text{key}(r).$$

Zpět k operaci INSERT

Zpět k operaci INSERT

Stačí nám tedy přidat vrchol v na první volné místo na poslední hladině, popř. vytvořit novou hladinu, a následně zavolat BUBBLEUP.

Zpět k operaci INSERT

Stačí nám tedy přidat vrchol v na první volné místo na poslední hladině, popř. vytvořit novou hladinu, a následně zavolat BUBBLEUP.

Algoritmus: Operace INSERT

Vstup: Nový vrchol v

- 1 $n \leftarrow n + 1$;
 - 2 **if** poslední hladina je plná **then**
 - 3 vlož v do nové hladiny co nejvíce vlevo
 - 4 **else**
 - 5 vlož v do poslední hladiny zleva
 - 6 Zavolej BUBBLEUP(v).
-

Operace **EXTRACTMIN** – odstranění minima

Operace **EXTRACTMIN** – odstranění minima

- Již víme, že přečtení minima zvládneme v konstantním čase (stačí se podívat na klíč v kořeni haldy a vrátit jej).

Operace **EXTRACTMIN** – odstranění minima

- Již víme, že přečtení minima zvládneme v konstantním čase (stačí se podívat na klíč v kořeni haldy a vrátit jej).
- Pokud budeme ale chtít odstranit minimum, bude situace již o trochu složitější.

Operace **EXTRACTMIN** – odstranění minima

- Již víme, že přečtení minima zvládneme v konstantním čase (stačí se podívat na klíč v kořeni haldy a vrátit jej).
- Pokud budeme ale chtít odstranit minimum, bude situace již o trochu složitější.

Musíme vyřešit dvě otázky:

Operace **EXTRACTMIN** – odstranění minima

- Již víme, že přečtení minima zvládneme v konstantním čase (stačí se podívat na klíč v kořeni haldy a vrátit jej).
- Pokud budeme ale chtít odstranit minimum, bude situace již o trochu složitější.

Musíme vyřešit dvě otázky:

- Co bude náš nový kořen?

Operace **EXTRACTMIN** – odstranění minima

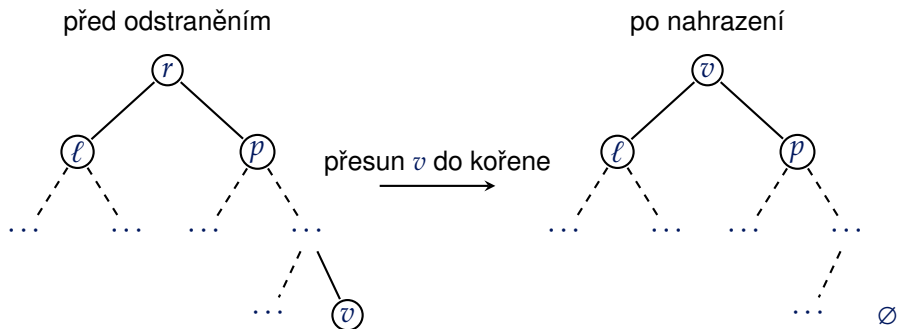
- Již víme, že přečtení minima zvládneme v konstantním čase (stačí se podívat na klíč v kořeni haldy a vrátit jej).
- Pokud budeme ale chtít odstranit minimum, bude situace již o trochu složitější.

Musíme vyřešit dvě otázky:

- Co bude náš nový kořen?
- Jak opravit haldu, aby platilo haldové uspořádání?

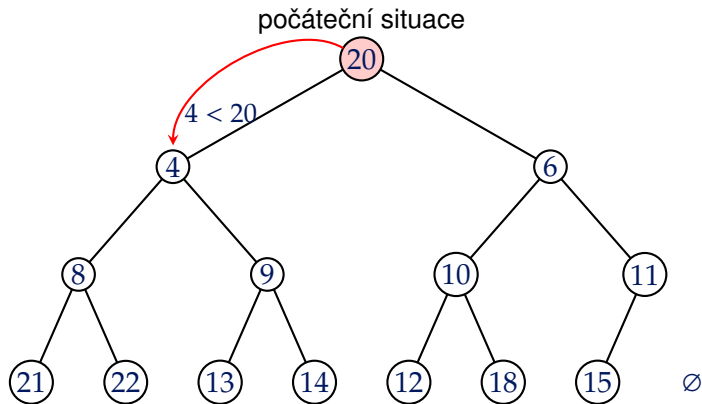
- Abychom zachovali tvar haldy, přemístíme **nejpravější vrchol z poslední hladiny** a umístíme jej na pozici kořene.
- Tím však může dojít k porušení haldového uspořádání.
- Idea opravy bude podobná jako v případě operace **INSERT**, ale nový kořen bude tentokrát „proublávat“ směrem dolů.
 - Pokud klíč v novém kořeni *v* je větší než klíč jedom z jeho synů, prohodíme je.
 - Pokud je hodnota klíče v novém kořeni větším než klíče v obou jeho synech, prohodíme jej s tím, jehož klíč je menší.
 - Pokud stále není splněno haldové uspořádání, tak postup opakujeme.

Nahrazení původního kořene



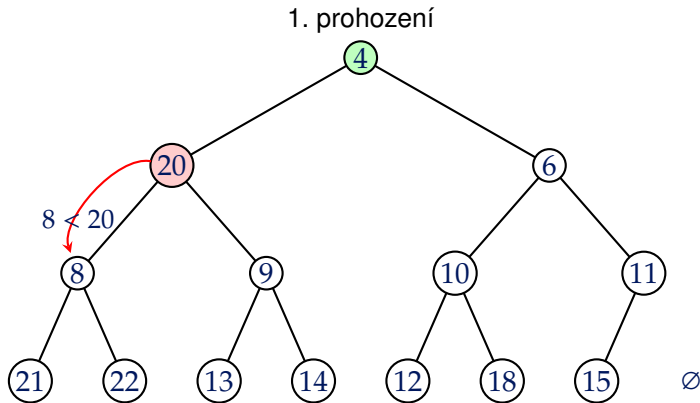
Procedura BUBBLEDOWN

1. krok



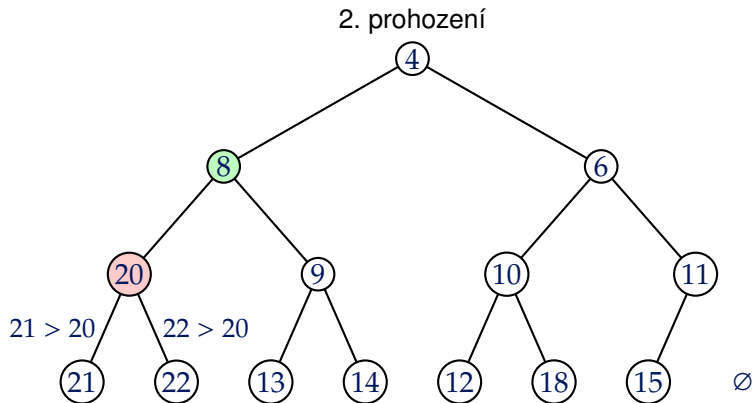
Procedura BUBBLEDOWN

2. krok



Procedura BUBBLEDOWN

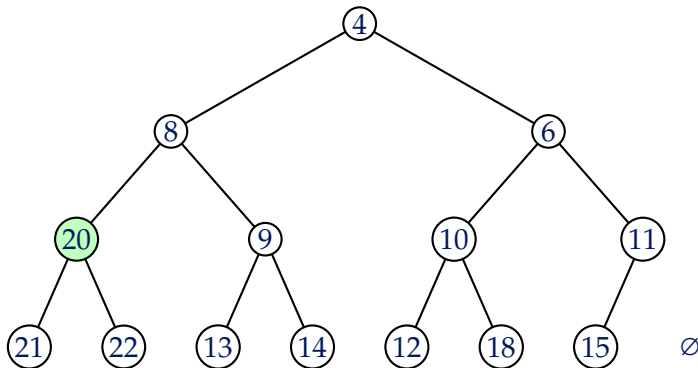
3. krok



Procedura BUBBLEDOWN

4. krok

výsledná minimová halda



Algoritmus: BUBBLEDOWN (oprava haldy směrem dolů)

Vstup: Vrchol v v minimové binární haldě

```
1 while  $v$  má alespoň jednoho syna do
2    $s \leftarrow$  syn vrcholu  $v$  s nejmenší hodnotou;
3   if  $\text{key}(v) \leq \text{key}(s)$  then
4     break;
5   prohod' klíče ve vrcholech  $v$  a  $s$ ;
6    $v \leftarrow s$ ;
```

Pseudokód BUBBLEDOWN

Algoritmus: BUBBLEDOWN (oprava haldy směrem dolů)

Vstup: Vrchol v v minimové binární haldě

```
1 while  $v$  má alespoň jednoho syna do
2    $s \leftarrow$  syn vrcholu  $v$  s nejmenší hodnotou;
3   if  $\text{key}(v) \leq \text{key}(s)$  then
4     break;
5   prohod' klíče ve vrcholech  $v$  a  $s$ ;
6    $v \leftarrow s$ ;
```

V případě maximové binární haldy bychom opět jen použili opačnou nerovnost:

$$\text{key}(v) \geq \text{key}(s).$$

Algoritmus: Operace EXTRACTMIN

- 1 Odeber kořen r ;
 - 2 $r \leftarrow$ nejpravější vrchol v na poslední hladině;
 - 3 Smaž vrchol v ;
 - 4 Zavolej BUBBLEDOWN(r)
-

Rozbor časové složitosti

Rozbor časové složitosti

- Rozebereme časovou složitost operací `INSERT` a `EXTRACTMIN`.

Rozbor časové složitosti

- Rozebereme časovou složitost operací **INSERT** a **EXTRACTMIN**.
- Operace **MIN** má zjevně složitost konstantní.

Rozbor časové složitosti

- Rozebereme časovou složitost operací **INSERT** a **EXTRACTMIN**.
- Operace **MIN** má zjevně složitost konstantní.
- Největší roli budou hrát procedury **BUBBLEUP** a **BUBBLEDOWN**. Zbylé operace stihneme v konstantním čase.

Rozbor časové složitosti

- Rozebereme časovou složitost operací **INSERT** a **EXTRACTMIN**.
- Operace **MIN** má zjevně složitost konstantní.
- Největší roli budou hrát procedury **BUBBLEUP** a **BUBBLEDOWN**. Zbylé operace stihneme v konstantním čase.

Každý vrchol „probublá“ v nejhorším případě
všemi hladinami.

Rozbor časové složitosti

- Rozebereme časovou složitost operací **INSERT** a **EXTRACTMIN**.
- Operace **MIN** má zjevně složitost konstantní.
- Největší roli budou hrát procedury **BUBBLEUP** a **BUBBLEDOWN**. Zbývající operace stihneme v konstantním čase.

Každý vrchol „probublá“ v nejhorším případě
všemi hladinami.

- Podle dokázaného tvrzení víme, že binární halda má celkově $\lceil \log_2(n + 1) \rceil$ hladin.

Rozbor časové složitosti

- Rozebereme časovou složitost operací **INSERT** a **EXTRACTMIN**.
- Operace **MIN** má zjevně složitost konstantní.
- Největší roli budou hrát procedury **BUBBLEUP** a **BUBBLEDOWN**. Zbylé operace stihneme v konstantním čase.

Každý vrchol „probublá“ v nejhorším případě
všemi hladinami.

- Podle dokázaného tvrzení víme, že binární halda má celkově $\lceil \log_2(n+1) \rceil$ hladin.
- Pak celková časová složitost činí

$$\begin{aligned} O(\log_2(n+1)) &= O(\log_2(n+n)) = O(\log_2 2 + \log_2 n) = O(\log_2 n) \\ &= O\left(\frac{1}{\log 2} \cdot \log n\right) = O(\log n) \end{aligned}$$

Rozdíl oproti poli

Vylepšili jsme časovou složitost z **lineární** na
logaritmickou!

Vylepšili jsme časovou složitost z **lineární** na **logaritmickou**!

Nicméně jak můžeme efektivně haldu implementovat?

Halda jako pole

Halda jako pole

Halda jako pole

- Binární haldu lze jistě implementovat jako graf.

Halda jako pole

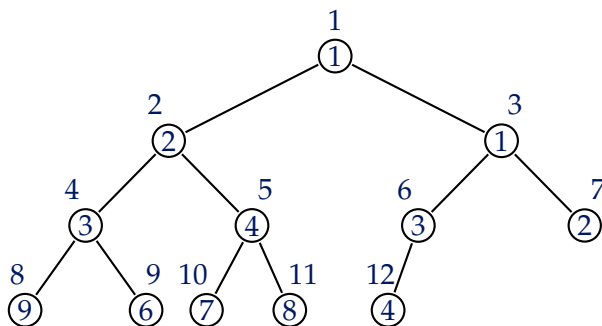
- Binární haldu lze jistě implementovat jako graf.
- Nicméně díky „pravidelnosti“ haldy (díky haldovému uspořádání a tvaru) lze využít jednodušší strukturu $\Rightarrow$ pole.

Halda jako pole

- Binární haldu lze jistě implementovat jako graf.
- Nicméně díky „pravidelnosti“ haldy (díky haldovému uspořádání a tvaru) lze využít jednodušší strukturu $\Rightarrow$ pole.
- Představme si, že vrcholy budeme číslovat od 1 do n , přičemž začínáme od kořene a postupujeme po hladinách zleva doprava.

Halda jako pole

- Binární haldu lze jistě implementovat jako graf.
- Nicméně díky „pravidelnosti“ haldy (díky haldovému uspořádání a tvaru) lze využít jednodušší strukturu $\Rightarrow$ **pole**.
- Představme si, že vrcholy budeme číslovat od 1 do n , přičemž začínáme od kořene a postupujeme po hladinách zleva doprava.

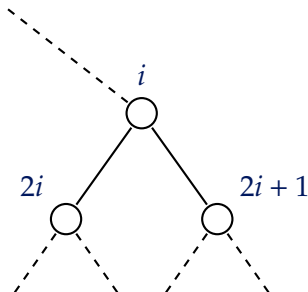


Indexy dětí a rodiče

Všimněme si, že když rodič má index i , pak jeho levý syn má index $2i$ a pravý syn $2i + 1$.

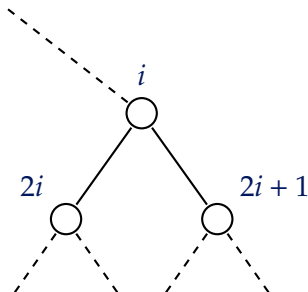
Indexy dětí a rodiče

Všimněme si, že když rodič má index i , pak jeho levý syn má index $2i$ a pravý syn $2i + 1$.



Indexy dětí a rodiče

Všimněme si, že když rodič má index i , pak jeho levý syn má index $2i$ a pravý syn $2i + 1$.



Resp. indexy dětí budou $2i + 1$ a $2i + 2$ v případě indexace od nuly.

Další operace

Změna klíče ve vrcholu

- Ještě se nabízí otázka, jak zařídit udržování haldy v případě, že budeme chtít upravovat klíče ve vrcholech.

- Ještě se nabízí otázka, jak zařídit udržování haldy v případě, že budeme chtít upravovat klíče ve vrcholech.
- K tomu se nám opět budou operace **BUBBLEUP** a **BUBBLEDOWN**.

Změna klíče ve vrcholu

- Ještě se nabízí otázka, jak zařídit udržování haldy v případě, že budeme chtít upravovat klíče ve vrcholech.
- K tomu se nám opět budou operace **BUBBLEUP** a **BUBBLEDOWN**.
- Opět uvažujme, že pracujeme s minimovou binární haldou (pro maximovou bude postup analogický).

Změna klíče ve vrcholu

- Ještě se nabízí otázka, jak zařídit udržování haldy v případě, že budeme chtít upravovat klíče ve vrcholech.
- K tomu se nám opět budou operace **BUBBLEUP** a **BUBBLEDOWN**.
- Opět uvažujme, že pracujeme s minimovou binární haldou (pro maximovou bude postup analogický).
- Budeme chtít přidat následující operace:

Změna klíče ve vrcholu

- Ještě se nabízí otázka, jak zařídit udržování haldy v případě, že budeme chtít upravovat klíče ve vrcholech.
- K tomu se nám opět budou operace **BUBBLEUP** a **BUBBLEDOWN**.
- Opět uvažujme, že pracujeme s minimovou binární haldou (pro maximovou bude postup analogický).
- Budeme chtít přidat následující operace:
 - **INCREASE**(v, x) = zvýší hodnotu klíče $\text{key}(v)$ o $x > 0$.

Změna klíče ve vrcholu

- Ještě se nabízí otázka, jak zařídit udržování haldy v případě, že budeme chtít upravovat klíče ve vrcholech.
- K tomu se nám opět budou operace **BUBBLEUP** a **BUBBLEDOWN**.
- Opět uvažujme, že pracujeme s minimovou binární haldou (pro maximovou bude postup analogický).
- Budeme chtít přidat následující operace:
 - **INCREASE**(v, x) = zvýší hodnotu klíče $\text{key}(v)$ o $x > 0$.
 - **DECREASE**(v, x) = sníží hodnotu klíče $\text{key}(v)$ o $x > 0$.

Operace INCREASE a DECREASE

Operace INCREASE a DECREASE

- Když budeme chtít zvýšit hodnotu klíče ve vrcholu v o hodnotu x , opět se musíme zamyslet, jak se může strom „pokazit“.

Operace INCREASE a DECREASE

- Když budeme chtít zvýšit hodnotu klíče ve vrcholu v o hodnotu x , opět se musíme zamyslet, jak se může strom „pokazit“.
- Pokud zvýšíme hodnotu klíče vrcholu v , pak v rodiči r hodnota klíče stále splňuje požadovanou nerovnost:

$$\text{key}(r) < \text{key}(v) < \text{key}(v) + x.$$

Operace INCREASE a DECREASE

- Když budeme chtít zvýšit hodnotu klíče ve vrcholu v o hodnotu x , opět se musíme zamyslet, jak se může strom „pokazit“.
- Pokud zvýšíme hodnotu klíče vrcholu v , pak v rodiči r hodnota klíče stále splňuje požadovanou nerovnost:

$$\text{key}(r) < \text{key}(v) < \text{key}(v) + x.$$

- Ovšem mohlo se pokazit haldové uspořádání v synech upraveného vrcholu v .

Operace INCREASE a DECREASE

- Když budeme chtít zvýšit hodnotu klíče ve vrcholu v o hodnotu x , opět se musíme zamyslet, jak se může strom „pokazit“.
- Pokud zvýšíme hodnotu klíče vrcholu v , pak v rodiči r hodnota klíče stále splňuje požadovanou nerovnost:

$$\text{key}(r) < \text{key}(v) < \text{key}(v) + x.$$

- Ovšem mohlo se pokazit haldové uspořádání v synech upraveného vrcholu v .

To však již umíme vyřešit procedurou **BUBBLEDOWN**! Stačí nám tedy zavolat **BUBBLEDOWN(v)** a jsme hotovi.

Operace INCREASE a DECREASE

- Když budeme chtít zvýšit hodnotu klíče ve vrcholu v o hodnotu x , opět se musíme zamyslet, jak se může strom „pokazit“.
- Pokud zvýšíme hodnotu klíče vrcholu v , pak v rodiči r hodnota klíče stále splňuje požadovanou nerovnost:

$$\text{key}(r) < \text{key}(v) < \text{key}(v) + x.$$

- Ovšem mohlo se pokazit haldové uspořádání v synech upraveného vrcholu v .

To však již umíme vyřešit procedurou **BUBBLEDOWN**! Stačí nám tedy zavolat **BUBBLEDOWN(v)** a jsme hotovi.

- Podobně budeme postupovat v případě operace **DECREASE** $\implies$ zavoláme **BUBBLEUP(v)**.

Složitost operací INCREASE a DECREASE

Složitost operací INCREASE a DECREASE

Je zjevné, že nejdéle potrvají procedury BUBBLEUP a BUBBLEDOWN, o nicž víme, že trvají $O(\log n)$.

Složitost operací INCREASE a DECREASE

Je zjevné, že nejdéle potrvají procedury BUBBLEUP a BUBBLEDOWN, o nicž víme, že trvají $O(\log n)$.

Tzn. i tyto operace mají **logaritmickou časovou složitost**.

Operace	Pole	Binární halda
MIN	$O(n)$	$O(\log n)$
MAX	$O(n)$	$O(\log n)$
INSERT	$O(1)$	$O(\log n)$
EXTRACTMIN	$O(n)$	$O(\log n)$
EXTRACTMAX	$O(n)$	$O(\log n)$

Operace	Pole	Binární halda
MIN	$O(n)$	$O(\log n)$
MAX	$O(n)$	$O(\log n)$
INSERT	$O(1)$	$O(\log n)$
EXTRACTMIN	$O(n)$	$O(\log n)$
EXTRACTMAX	$O(n)$	$O(\log n)$

Vidíme, že jsme si **polepšili** v případě **hledání minima/maxima** a jeho **odstranění**, ale na druhou stranu jsme si **pohoršili** při **vkládání nového vrcholu**.

- MAREŠ, Martin a VALLA, Tomáš. *Průvodce labyrintem algoritmů*. Druhé vydání. CZ.NIC. Praha: CZ.NIC, z.s.p.o., 2022. ISBN 978-80-88168-63-8.
- CORMEN, Thomas H.; LEISERSON, Charles Eric; RIVEST, Ronald L. a STEIN, Clifford. *Introduction to algorithms*. Fourth edition. Cambridge: The MIT Press, 2022. ISBN 978-0-262-04630-5.